LAB REPORT - 3

REST API, query parameters, validation, debugging

Student Name	ODDEPELLY SAIKIRAN	**Additional Info**	
Student Registration Number	252U1R5116	**Class & Section**	313 AI DE 2B
Study Level : UG/PG/Doctorate	UG	**Year & Term**	2 & 2
Subject Name	Backend Development Fundamentals		
Teacher / Faculty Name	Mr Yugandhar Gurjalwar		
Name of the Assessment	LAB REPORT - 3		
Date of Submission	6/11/2026		

Objective & Scope:

Transform SQLite CRUD operations into an enterprise-grade RESTful API featuring query parameter handling (search LIKE, multi-column sorting, price range BETWEEN filter, limit/offset pagination), request body validation, consistent JSON responses, robust error handling (400 vs 404), and systematic debugging practices.

REST API Specification Table:

Method	Endpoint	Parameters / Body	Status	Description
GET	/books/	search_q, order_by, order, min_price, max_price, limit, offset	200 OK	Get books with filters
GET	/books/:bookId/	bookId in URL path	200 / 404	Get single book by ID
POST	/books/	{"title", "author", "price", "pages"}	201 / 400	Add new book
PUT	/books/:bookId/	{"title", "author", "price", "pages"}	200 / 400 / 404	Update book details
DELETE	/books/:bookId/	bookId in URL path	200 / 404	Delete book by ID

1. Database Initialization & Seeding Script (setupDb.js)

Strict clean code implementation with zero comments and parameterized queries (?):

```
const path = require("path");
const sqlite3 = require("sqlite3");
const { open } = require("sqlite");

const dbPath = path.join(__dirname, "books.db");

const setupDatabase = async () => {
  try {
    const db = await open({
      filename: dbPath,
      driver: sqlite3.Database
    });

    await db.exec(`
      DROP TABLE IF EXISTS book;
      CREATE TABLE book (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        author TEXT NOT NULL,
        price INTEGER NOT NULL,
        pages INTEGER NOT NULL
      );
    `);

    const books = [
      { title: "To Kill a Mockingbird", author: "Harper Lee", price: 399, pages: 336 },
      { title: "1984", author: "George Orwell", price: 299, pages: 328 },
      { title: "The Great Gatsby", author: "F. Scott Fitzgerald", price: 249, pages: 180 },
      { title: "The Catcher in the Rye", author: "J.D. Salinger", price: 349, pages: 240 },
      { title: "Pride and Prejudice", author: "Jane Austen", price: 199, pages: 432 },
      { title: "The Hobbit", author: "J.R.R. Tolkien", price: 499, pages: 310 },
      { title: "Fahrenheit 451", author: "Ray Bradbury", price: 279, pages: 256 },
      { title: "Jane Eyre", author: "Charlotte Bronte", price: 319, pages: 532 },
      { title: "Animal Farm", author: "George Orwell", price: 199, pages: 144 },
      { title: "Brave New World", author: "Aldous Huxley", price: 359, pages: 288 }
    ];

    for (const book of books) {
      await db.run(
        `INSERT INTO book (title, author, price, pages) VALUES (?, ?, ?, ?);`,
        [book.title, book.author, book.price, book.pages]
      );
    }

    console.log("Database initialized and 10 records seeded successfully.");
    await db.close();
  } catch (error) {
    console.error("Error setting up database:", error.message);
    process.exit(1);
  }
};

setupDatabase();
```

2. Configuration (package.json)

```
{
  "name": "lab-03",
  "version": "1.0.0",
  "description": "Lab 3: REST API, query parameters, validation, debugging - Backend Development Fundamentals",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js",
    "setup-db": "node setupDb.js"
  },
  "keywords": [
    "express",
    "sqlite",
    "sqlite3",
    "rest-api",
    "validation",
    "query-parameters"
  ],
  "author": "ODDEPELLY SAIKIRAN",
  "license": "ISC",
  "dependencies": {
    "express": "^5.3.0",
    "sqlite": "^5.1.1",
    "sqlite3": "^6.0.1"
  },
  "devDependencies": {
```

3. Express Application & REST Endpoints (app.js - Part 1)

```
const express = require("express");
const path = require("path");
const { open } = require("sqlite");
const sqlite3 = require("sqlite3");

const app = express();
app.use(express.json());

const dbPath = path.join(__dirname, "books.db");
let db = null;

const initializeDBAndServer = async () => {
  try {
    db = await open({
      filename: dbPath,
      driver: sqlite3.Database
    });

    await db.exec(`
      CREATE TABLE IF NOT EXISTS book (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        author TEXT NOT NULL,
        price INTEGER NOT NULL,
        pages INTEGER NOT NULL
      );
    `);

    const PORT = process.env.PORT || 3000;
    app.listen(PORT, () => {
      console.log(`Server Running at http://localhost:${PORT}/`);
    });
  } catch (error) {
    console.error(`DB Error: ${error.message}`);
    process.exit(1);
  }
};

app.get("/", (req, res) => {
  res.status(200).json({
    message: "Lab 3: REST API, query parameters, validation, debugging",
    endpoints: {
      getBooks: "GET /books/?search_q=&order_by=&order=&limit=&offset=&min_price=&max_price=",
      getBookById: "GET /books/:bookId/",
      addBook: "POST /books/",
      updateBook: "PUT /books/:bookId/",
      deleteBook: "DELETE /books/:bookId/"
    }
  });
});

app.get(["/books", "/books/"], async (req, res) => {
  try {
    const {
      search_q = "",
      order_by = "id",
      order = "ASC",
      limit = 10,
      offset = 0,
      min_price,
      max_price
    } = req.query;

    const validColumns = ["id", "title", "author", "price", "pages"];
    const validOrderBy = validColumns.includes(order_by.toLowerCase())
      ? order_by.toLowerCase()
      : "id";
    const validOrder = order.toUpperCase() === "DESC" ? "DESC" : "ASC";
    const parsedLimit = parseInt(limit, 10) > 0 ? parseInt(limit, 10) : 10;
    const parsedOffset = parseInt(offset, 10) >= 0 ? parseInt(offset, 10) : 0;

    let query = `SELECT * FROM book WHERE title LIKE ?`;
    const params = [`%${search_q}%`];

    if (min_price !== undefined && max_price !== undefined && min_price !== "" && max_price !== "") {
      query += ` AND price BETWEEN ? AND ?`;
      params.push(Number(min_price), Number(max_price));
    } else if (min_price !== undefined && min_price !== "") {
      query += ` AND price >= ?`;
      params.push(Number(min_price));
    } else if (max_price !== undefined && max_price !== "") {
      query += ` AND price <= ?`;
      params.push(Number(max_price));
    }
```

3. Express Application & REST Endpoints (app.js - Part 2)

```
    query += ` ORDER BY ${validOrderBy} ${validOrder} LIMIT ? OFFSET ?;`;
    params.push(parsedLimit, parsedOffset);

    const books = await db.all(query, params);
    res.status(200).json(books);
  } catch (error) {
    res.status(500).json({ message: "Internal Server Error" });
  }
});

app.get(["/books/:bookId", "/books/:bookId/"], async (req, res) => {
  try {
    const { bookId } = req.params;
    const book = await db.get(`SELECT * FROM book WHERE id = ?;`, [bookId]);
    if (!book) {
      return res.status(404).json({ message: "Book Not Found" });
    }
    res.status(200).json(book);
  } catch (error) {
    res.status(500).json({ message: "Internal Server Error" });
  }
});

app.post(["/books", "/books/"], async (req, res) => {
  try {
    const { title, author, price, pages } = req.body;

    if (!title || typeof title !== "string" || title.trim() === "") {
      return res.status(400).json({ message: "Invalid or missing title" });
    }
    if (!author || typeof author !== "string" || author.trim() === "") {
      return res.status(400).json({ message: "Invalid or missing author" });
    }
    if (
      price === undefined ||
      price === null ||
      isNaN(price) ||
      Number(price) < 0 ||
      (typeof price === "string" && price.trim() === "")
    ) {
      return res.status(400).json({ message: "Invalid or non-numeric price" });
    }

    const parsedPages =
      pages !== undefined && !isNaN(pages) && Number(pages) > 0
        ? Number(pages)
        : 0;

    const result = await db.run(
      `INSERT INTO book (title, author, price, pages) VALUES (?, ?, ?, ?);`,
      [title.trim(), author.trim(), Number(price), parsedPages]
    );

    res.status(201).json({
      bookId: result.lastID,
      message: "Book Added Successfully"
    });
  } catch (error) {
    res.status(500).json({ message: "Internal Server Error" });
  }
});

app.put(["/books/:bookId", "/books/:bookId/"], async (req, res) => {
  try {
    const { bookId } = req.params;
    const existingBook = await db.get(`SELECT * FROM book WHERE id = ?;`, [bookId]);
    if (!existingBook) {
      return res.status(404).json({ message: "Book Not Found" });
    }

    const { title, author, price, pages } = req.body;

    if (!title || typeof title !== "string" || title.trim() === "") {
      return res.status(400).json({ message: "Invalid or missing title" });
    }
    if (!author || typeof author !== "string" || author.trim() === "") {
      return res.status(400).json({ message: "Invalid or missing author" });
    }
    if (
      price === undefined ||
      price === null ||
      isNaN(price) ||
      Number(price) < 0 ||
      (typeof price === "string" && price.trim() === "")
    ) {
      return res.status(400).json({ message: "Invalid or non-numeric price" });
    }

    const parsedPages =
      pages !== undefined && !isNaN(pages) && Number(pages) > 0
        ? Number(pages)
        : existingBook.pages;
```

Output 1: Default GET /books/ (Default Query Parameters)

Postman - GET /books/ (Default Query Parameters)

Collections
Lab 03 - REST API
GET /books/
GET ?search_q=
GET ?min_price=
GET ?order_by=
GET /books/:id/
POST /books/
PUT /books/:id/
DEL /books/:id/

Student:
ODDEPELLY SAIKIRAN
252U1R5116
313 AI DE 2B
Backend Dev Fundamentals

GET http://localhost:3000/books/ Send

Params Authorization Headers (2) Body

Response Status: 200 OK Time: 16 ms Size: 2.45 KB


```
[
  {
    "id":1,
    "title":"To Kill a Mockingbird",
    "author":"Harper Lee",
    "price":399,
    "pages":336
  },
  {
    "id":2,
    "title":"1984",
    "author":"George Orwell",
    "price":299,
    "pages":328
  },
  {
    "id":3,
    "title":"The Great Gatsby",
    "author":"F. Scott Fitzgerald",
    "price":249,
    "pages":180
  },
  {
    "id":4,
    "title":"The Catcher in the Rye",
    "author":"J.D. Salinger",
    "price":349,
```


Output 2: GET /books/?search_q=the (Partial Title Matching with LIKE)

Postman - GET /books/?search_q=the (Partial Title Match)

Collections
Lab 03 - REST API
GET /books/
GET ?search_q=
GET ?min_price=
GET ?order_by=
GET /books/:id/
POST /books/
PUT /books/:id/
DEL /books/:id/

Student:
ODDEPELLY SAIKIRAN
252U1R5116
313 AI DE 2B
Backend Dev Fundamentals

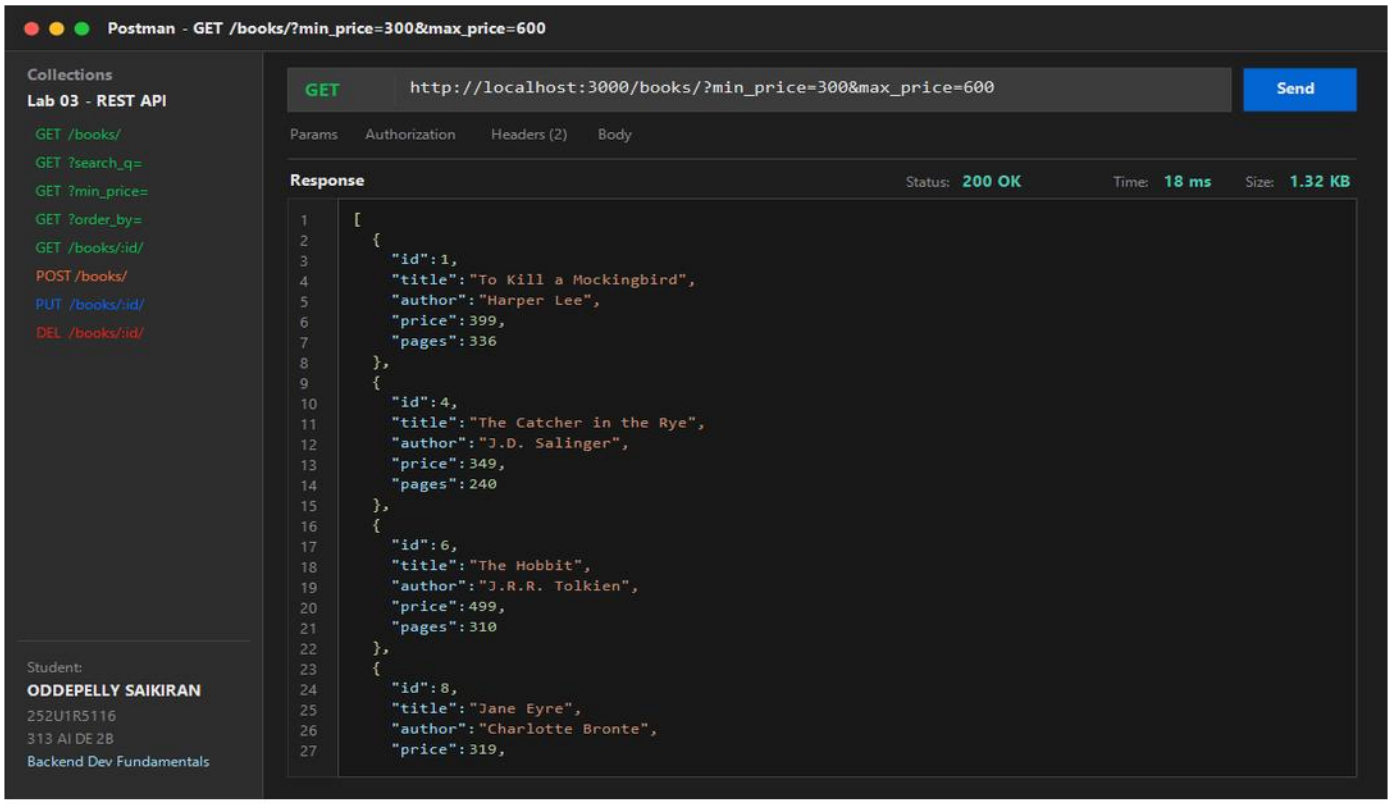
GET http://localhost:3000/books/?search_q=the Send

Params Authorization Headers (2) Body

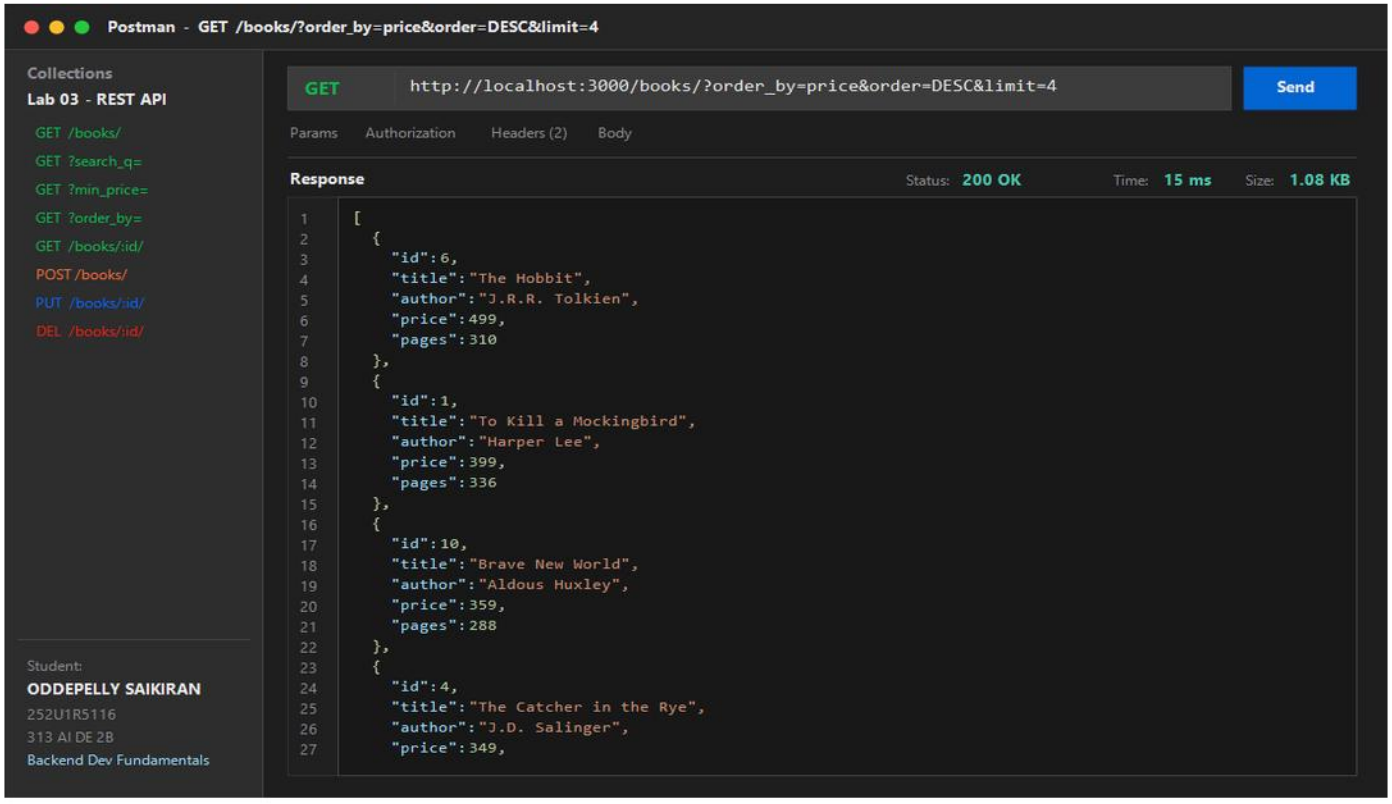
Response Status: 200 OK Time: 14 ms Size: 840 B


```
[
  {
    "id":3,
    "title":"The Great Gatsby",
    "author":"F. Scott Fitzgerald",
    "price":249,
    "pages":180
  },
  {
    "id":4,
    "title":"The Catcher in the Rye",
    "author":"J.D. Salinger",
    "price":349,
    "pages":240
  },
  {
    "id":6,
    "title":"The Hobbit",
    "author":"J.R.R. Tolkien",
    "price":499,
    "pages":310
  }
]
```

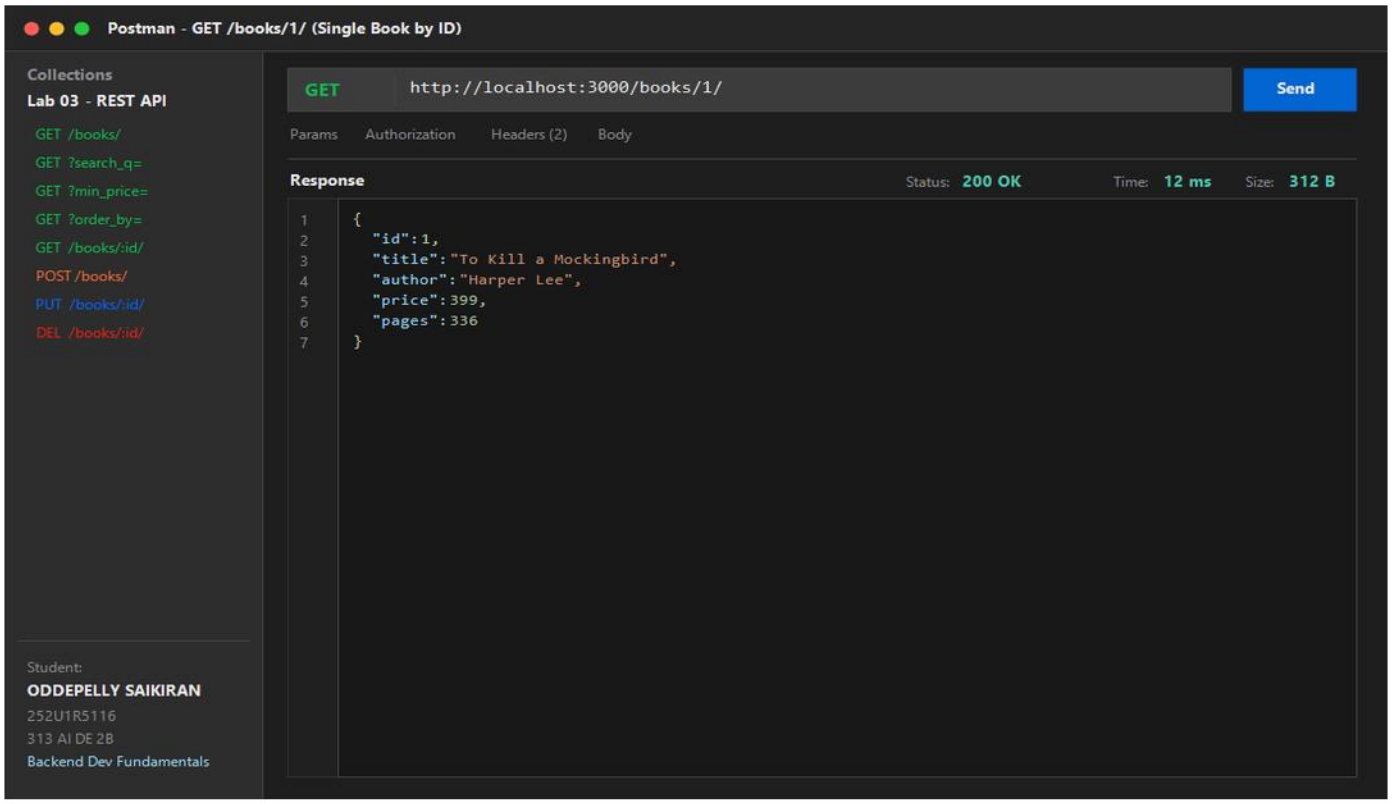
Output 3: GET /books/?min_price=300&max_price=600 (Price Range Filter)



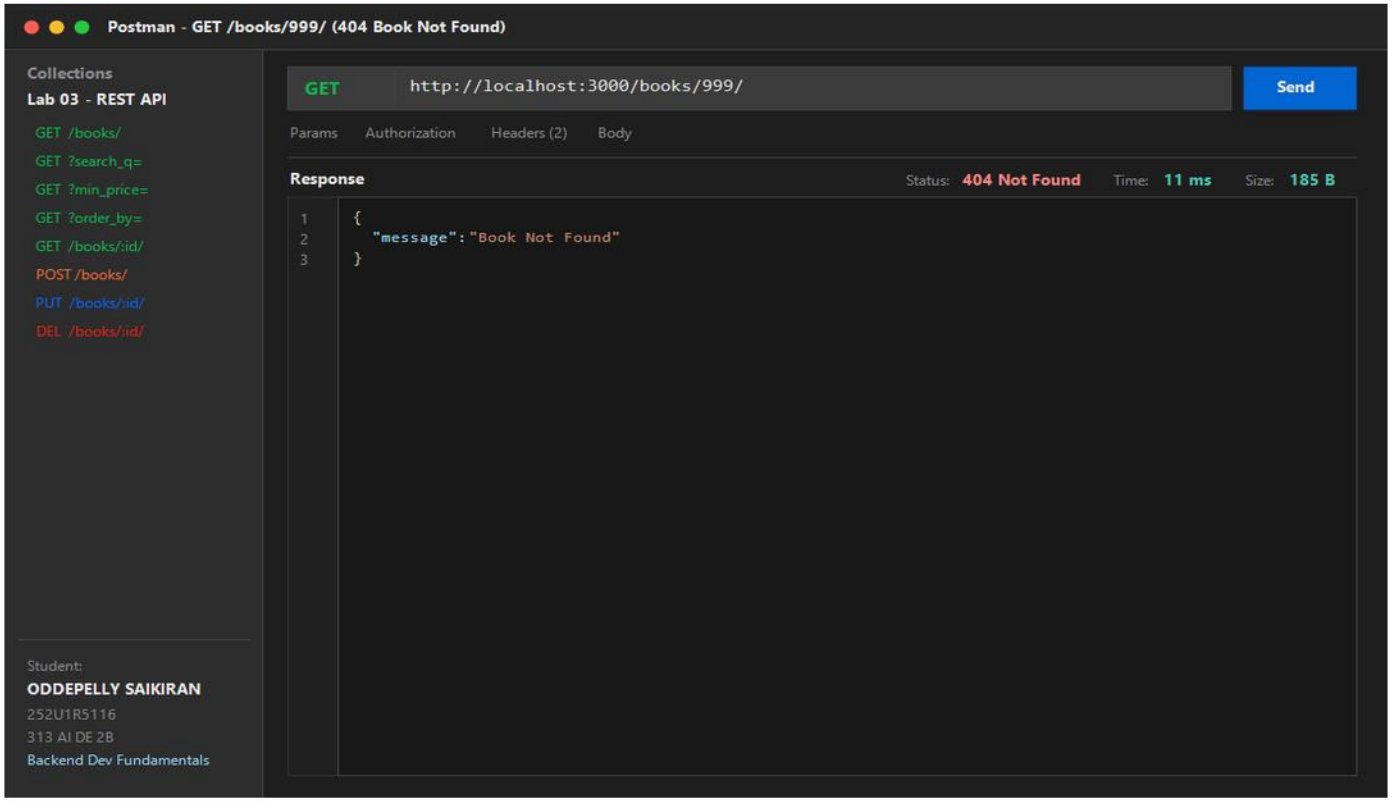
Output 4: GET /books/?order_by=price&order=DESC&limit=4 (Sorting & Pagination)



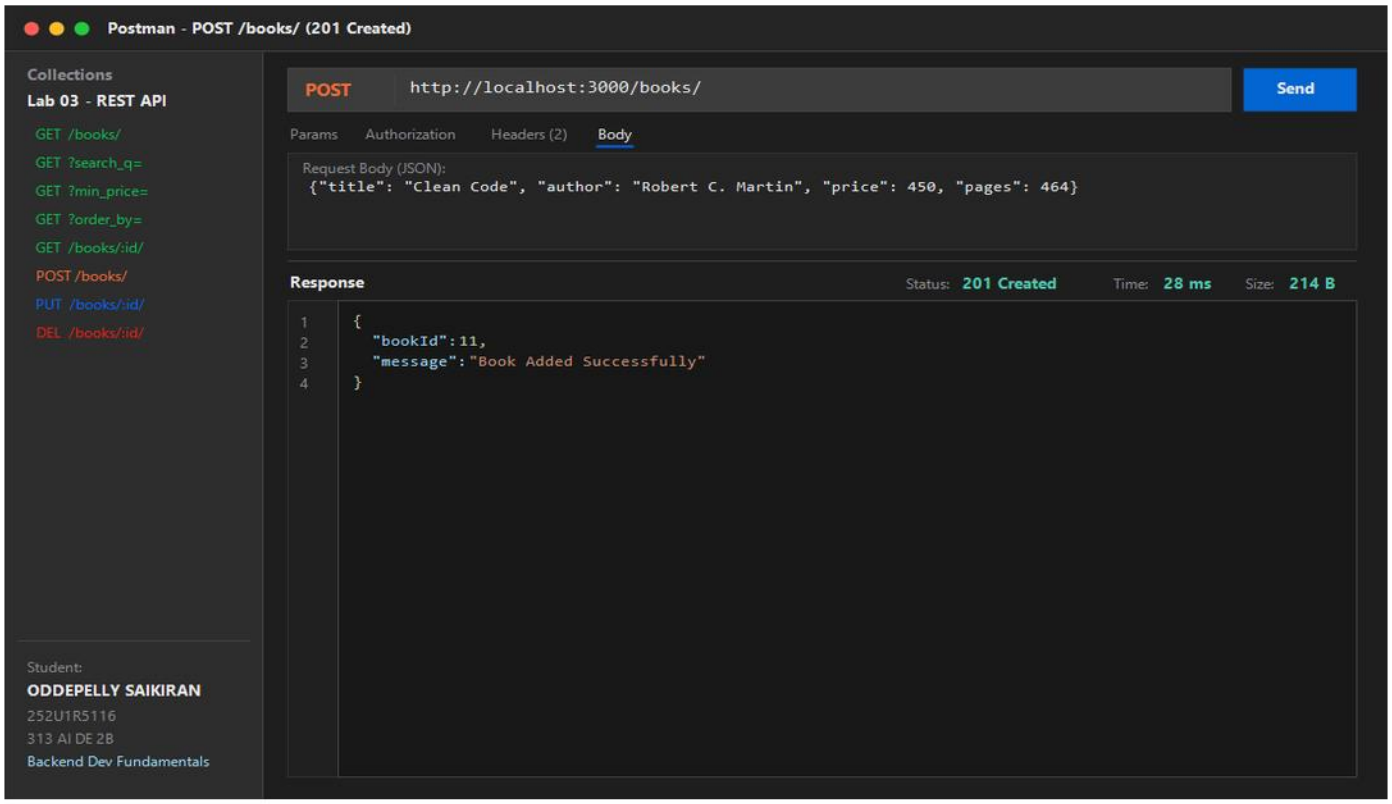
Output 5: GET /books/1/ (Single Book by Primary Key ID)



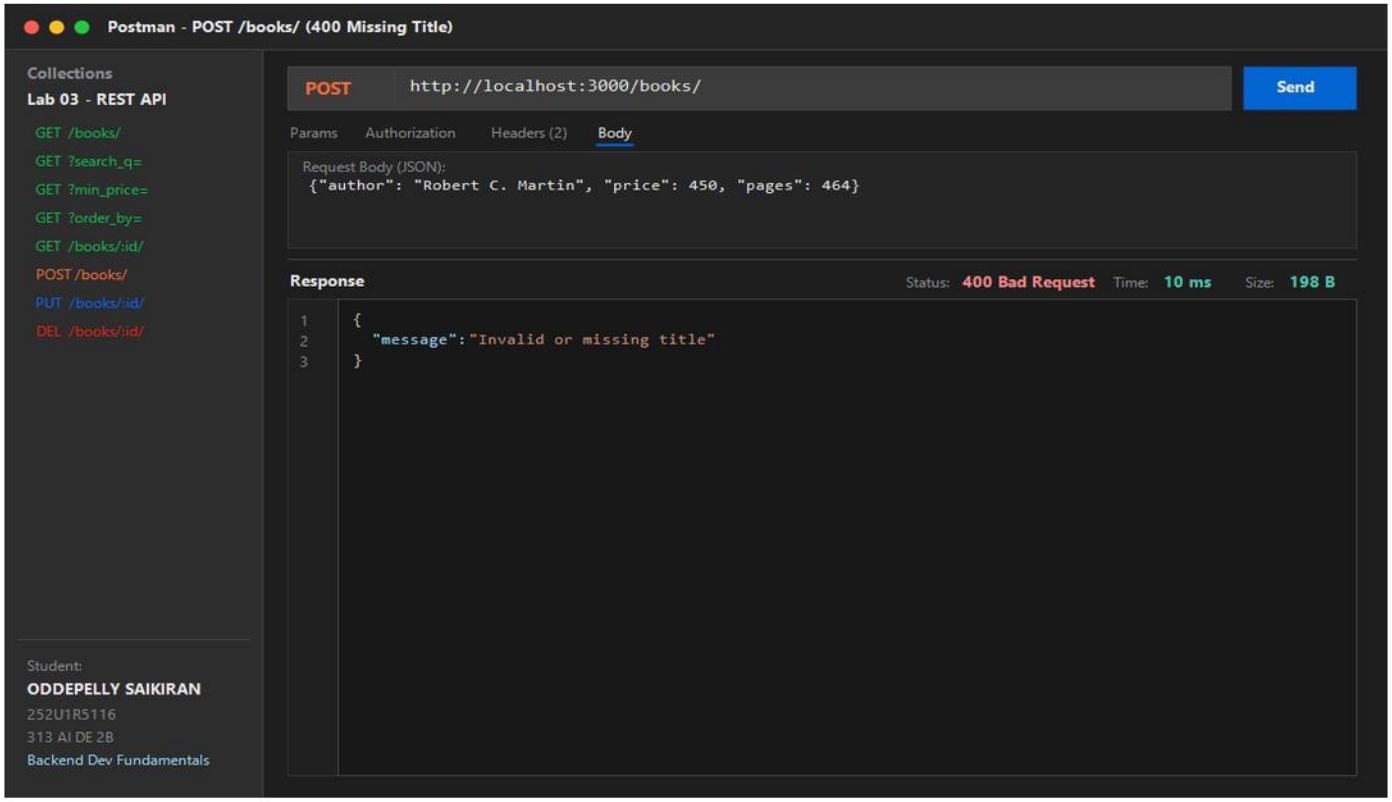
Output 6: GET /books/999/ (404 Book Not Found Handling)



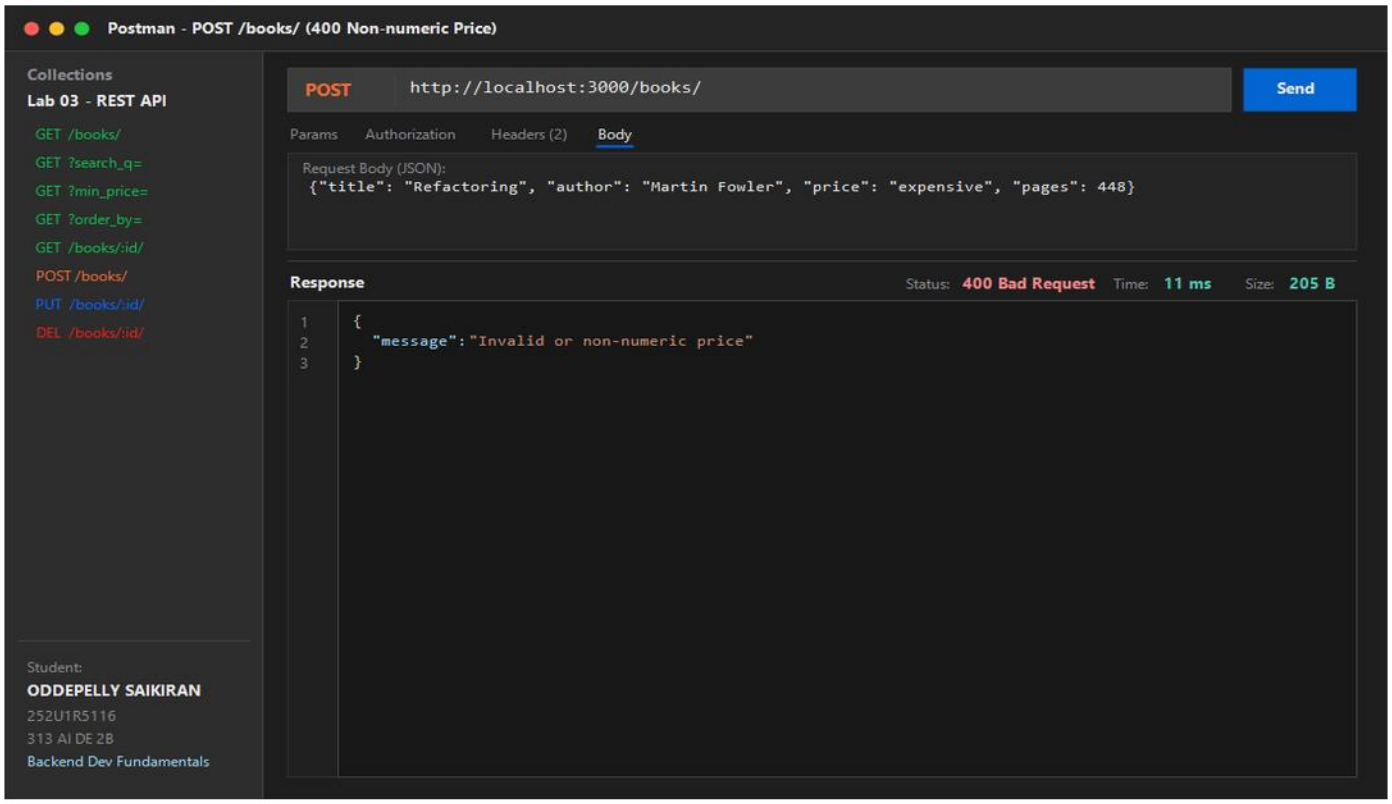
Output 7: POST /books/ Success (201 Created with Generated ID)



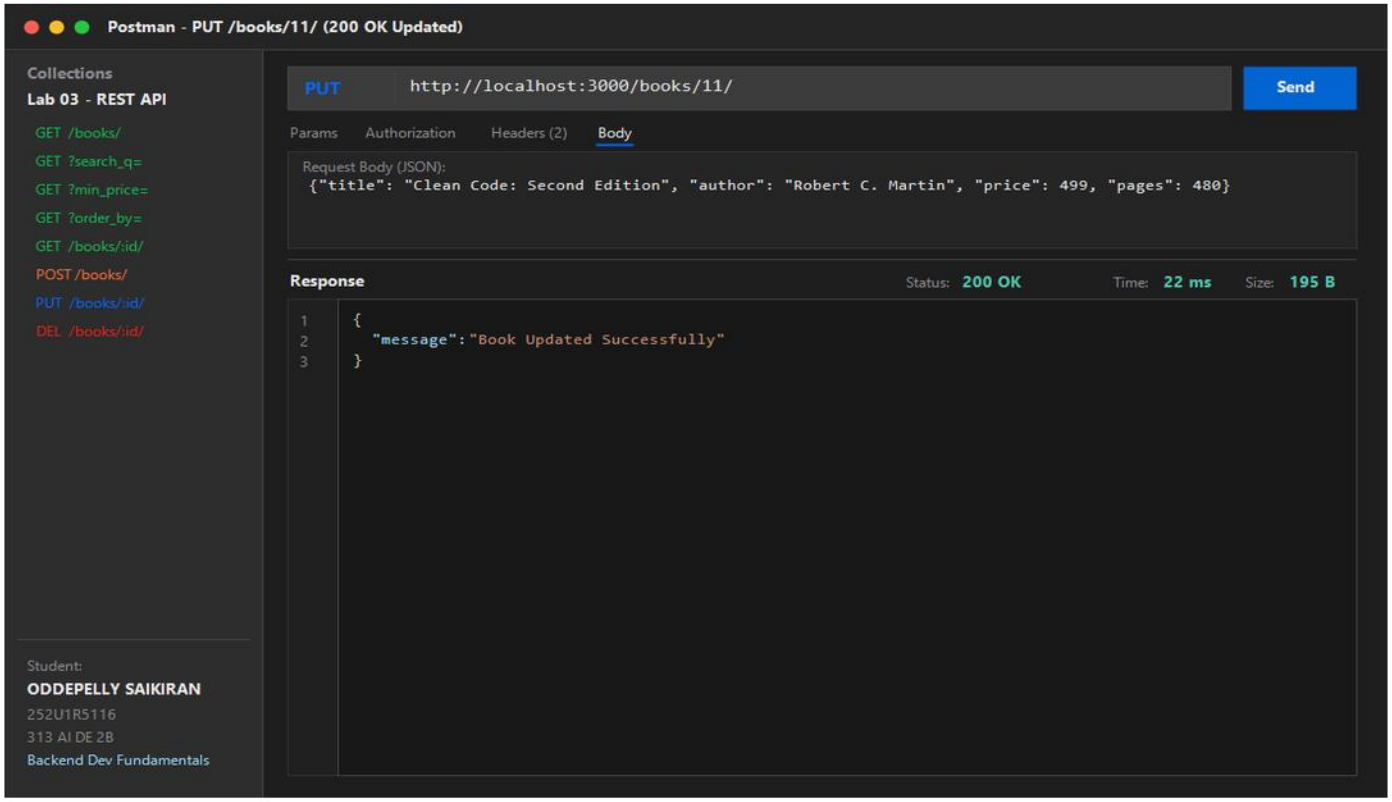
Output 8: POST /books/ Missing Title Validation Error (400 Bad Request)



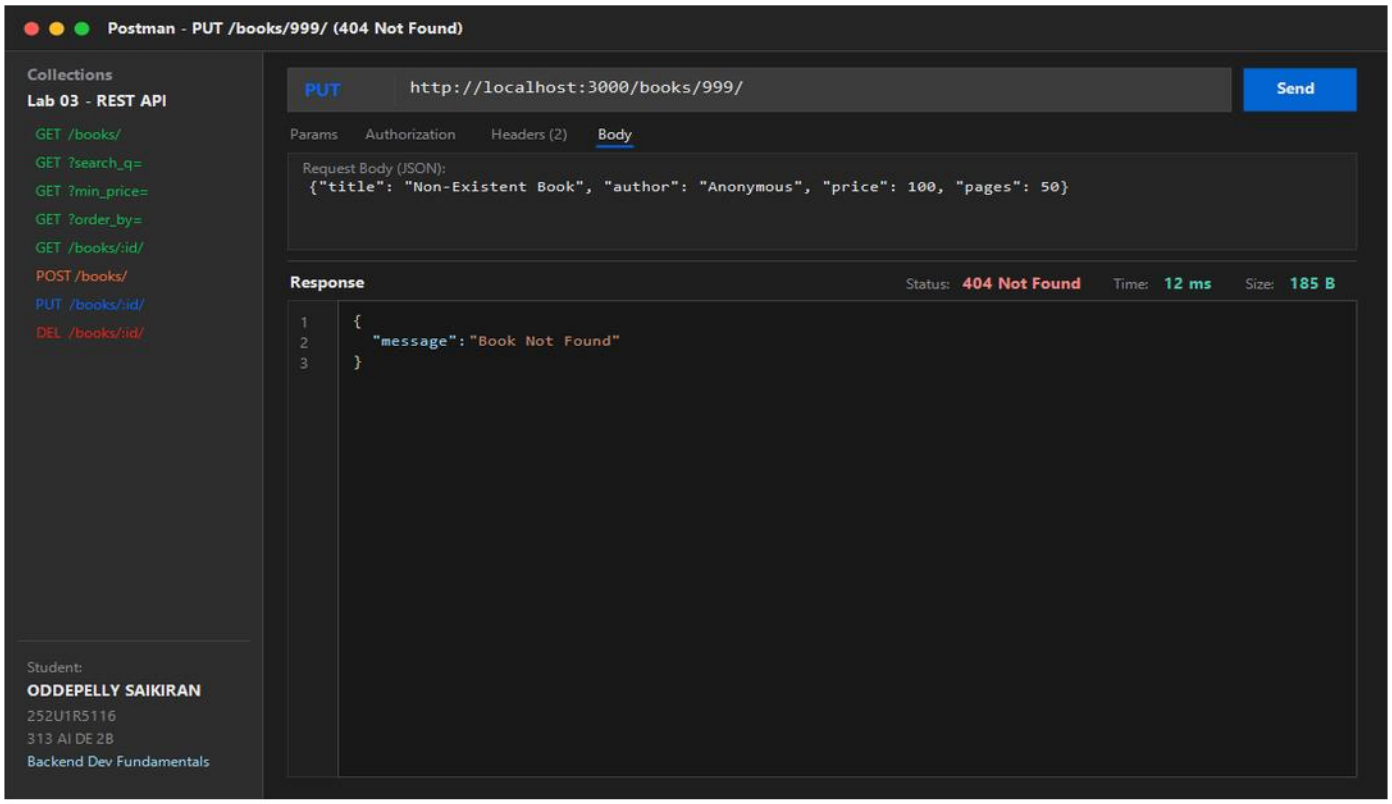
Output 9: POST /books/ Non-Numeric Price Validation Error (400 Bad Request)



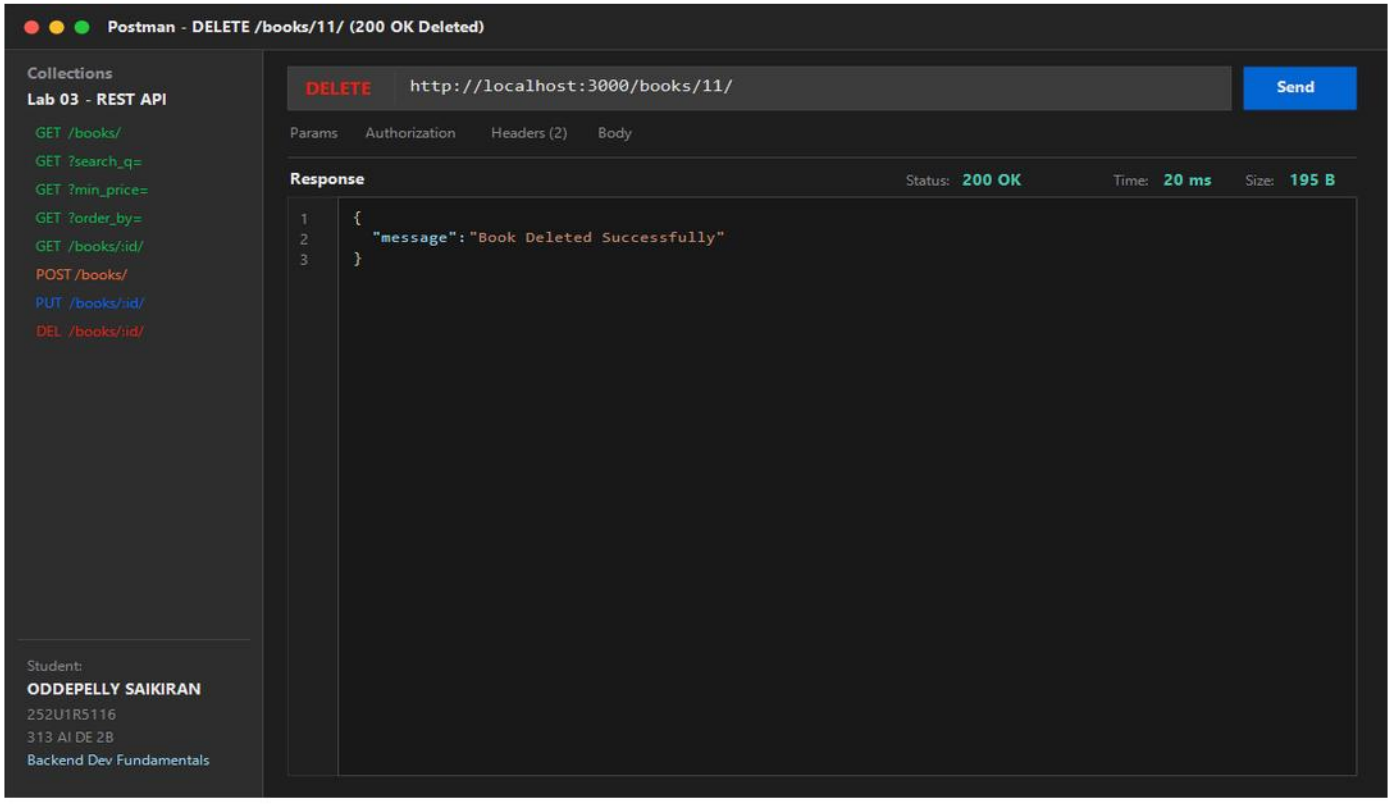
Output 10: PUT /books/11/ Success (200 OK Record Updated)



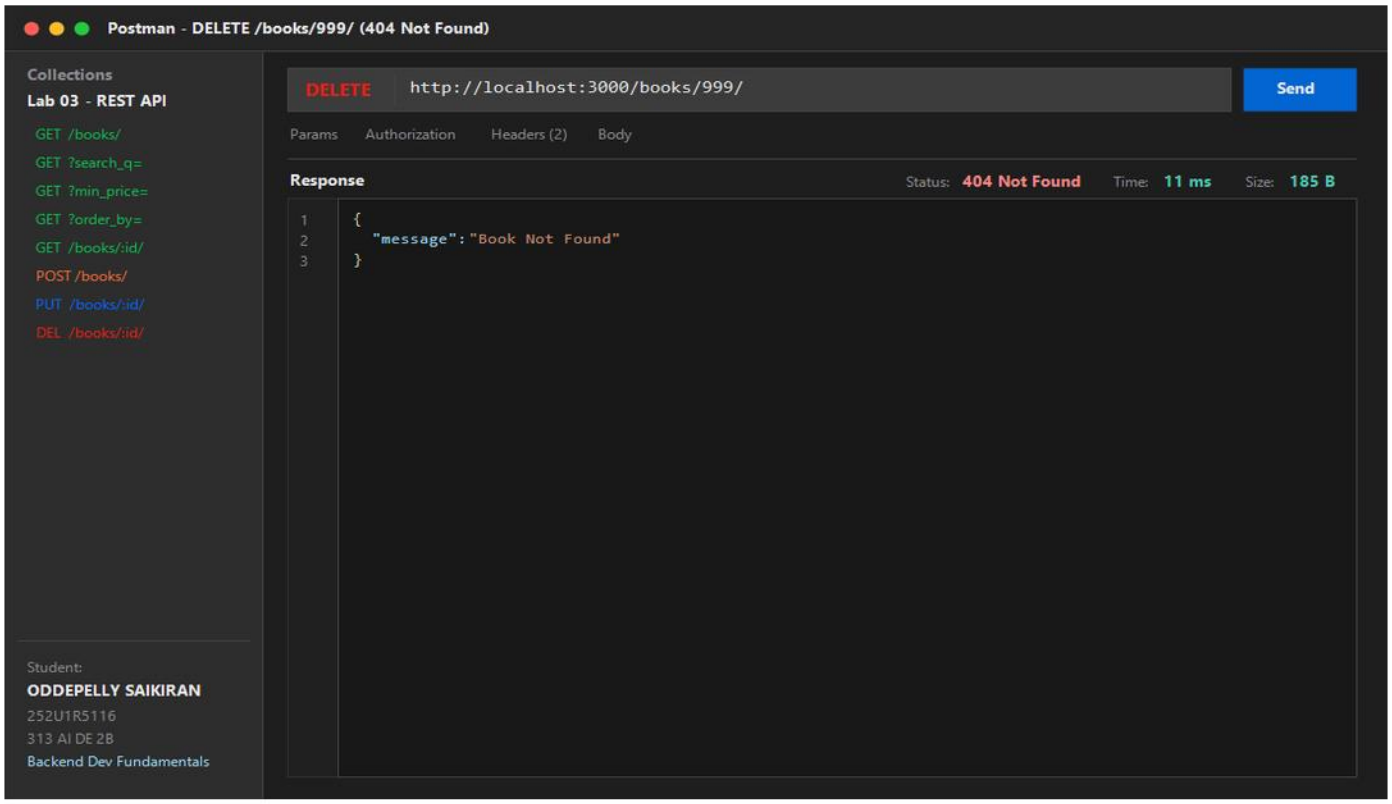
Output 11: PUT /books/999/ (404 Not Found Handling)



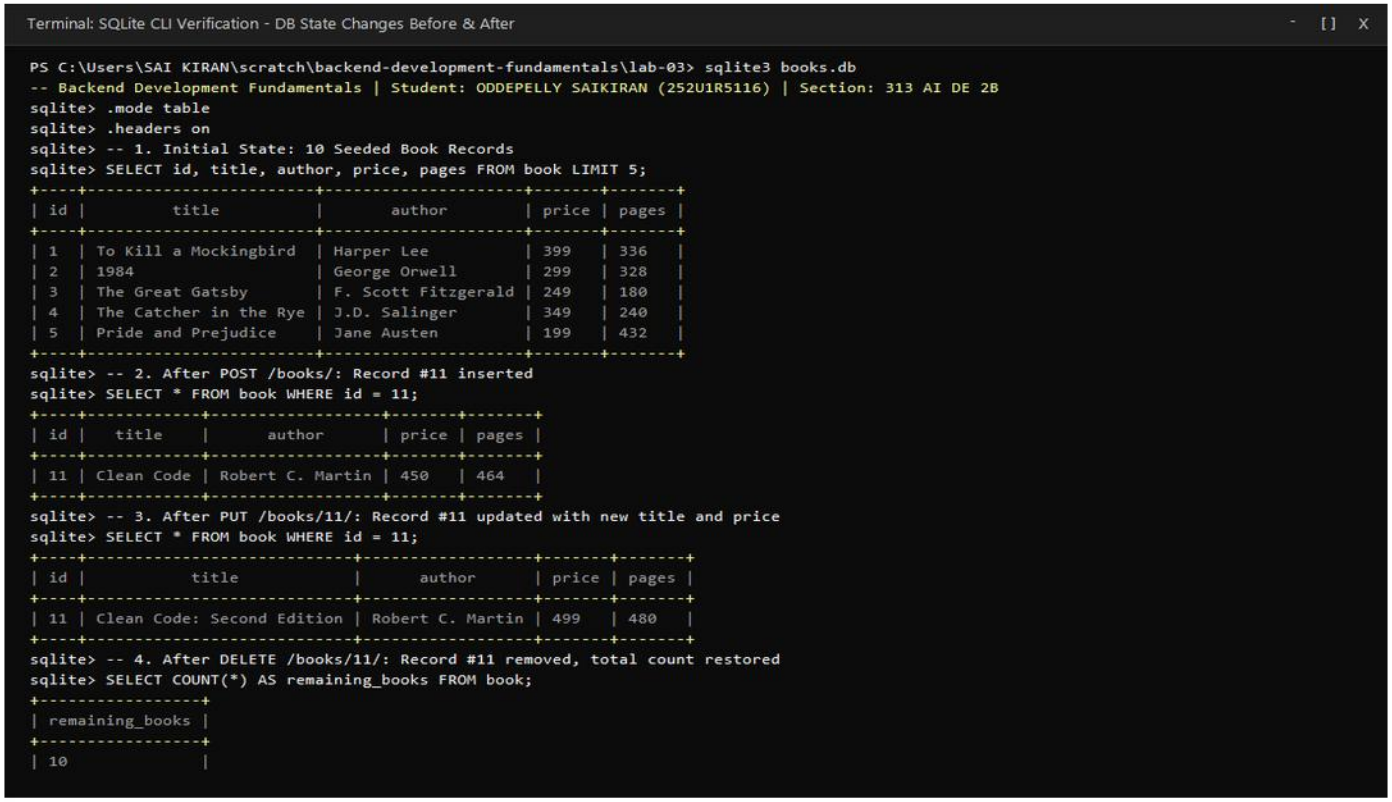
Output 12: DELETE /books/11/ Success (200 OK Record Deleted)



Output 13: DELETE /books/999/ (404 Not Found Handling)



Output 14: Terminal SQLite CLI Verification (Database State Changes Before & After)



Viva Questions & Technical Answers

Q1: What is the difference between a query parameter and a path parameter?

Path parameters (/books/:bookId/) are part of the resource identifier URL path. Used to uniquely identify a specific resource entity (e.g. book with ID 1) and are mandatory. Query parameters (?order_by=price&limit=4) appear after '?' as key-value pairs used for optional operations like search, filtering, sorting, and pagination with default fallback values.

Q2: What does stateless mean in RESTful API architecture?

Statelessness means that every request from the client contains all the information needed for the server to process it. The server does not retain client session context between requests. This enables high horizontal scalability, cacheability, and fault tolerance.

Q3: Which HTTP status code should be returned for bad input versus a missing record?

HTTP 400 Bad Request is returned for malformed or invalid client input (missing required title/author or non-numeric/negative price).
HTTP 404 Not Found is returned when a valid identifier is queried or modified, but no matching record exists in the database.

Debugging Notes (Common Backend Exercises & Solutions)

1. Port Conflict (EADDRINUSE)

Occurs when port 3000 is occupied by a lingering process. Solved using 'netstat -ano | findstr :3000' and 'taskkill /PID <PID> /F', or configurable 'process.env.PORT || 3000'.

2. Wrong Import Path (MODULE_NOT_FOUND)

Occurs from relative path typos. Solved by verifying installed packages in package.json and exact directory traversal paths.

3. Table-Name Typo (SQLITE_ERROR: no such table)

Caused by mismatches such as 'SELECT * FROM books' instead of singular 'book'. Solved by standardizing table names across schema and SQL queries.

4. Missing await (Unhandled Async Execution / Pending Promise)

Occurs when omitting 'await' before asynchronous SQLite calls (db.all, db.get, db.run), causing Node to serialize Promise { <pending> } prematurely. Solved by declaring route har